

TP Intégration continue

• Contexte de la Mission

Vous venez d'être recruté en tant qu'Ingénieur DevOps au sein d'une startup. L'équipe de développement a produit un microservice d'authentification en Java (Maven).

Malheureusement, le code a été écrit dans l'urgence, il contient des failles de sécurité et des mauvaises pratiques.

Votre objectif :

1. Mettre en place un pipeline d'Intégration Continue avec Jenkins.
2. Interconnecter SonarQube pour auditer ce code automatiquement.
3. Corriger les failles identifiées pour passer le *Quality Gate*.
4. *(Bonus)* Conteneuriser l'application avec Docker pour préparer son déploiement.

• Livrables et Évaluation

Pour valider cette session technique, votre travail sera évalué sur deux axes : un compte rendu écrit et une restitution orale courte. Mettez-vous dans la peau d'un Ingénieur DevOps qui justifie son audit technique auprès de son équipe.

1. Le Compte Rendu (Format PDF)

Ce document doit être concis, visuel et retracer chronologiquement votre démarche de résolution de problèmes. Il doit impérativement contenir :

- L'état des lieux initial : Les preuves des premiers échecs (captures d'écran du pipeline Jenkins au rouge et de l'audit SonarQube avec le détail des vulnérabilités).
- Le journal des corrections : Une liste claire expliquant chaque "Code Smell" ou faille identifié, et la modification exacte apportée dans le code Java pour le résoudre.
- Le résultat final : Les preuves du bon fonctionnement de votre intégration (capture du Quality Gate au vert sur SonarQube et du build Jenkins en succès).
- *(Si réalisé)* La preuve de déploiement : Une capture de la commande docker ps prouvant que votre conteneur est fonctionnel.

2. La Présentation Orale (5 à 10 minutes)

Vous viendrez présenter votre travail au tableau. L'objectif n'est pas de lire le compte rendu, mais de démontrer que vous avez compris les enjeux techniques :

- Analyse de sécurité : Expliquez pourquoi le mot de passe en dur était une faille critique et détaillez votre stratégie pour la corriger.

- **Qualité du code : Justifiez vos corrections**
- **Bilan CI/CD : Concluez sur la manière dont ce pipeline protège désormais l'application des futures erreurs de l'équipe.**

Grille d'évaluation : Compte Rendu Écrit (sur 20 points)

Critère d'évaluation	Description des attentes	Points
Pipeline Jenkins	Le Jenkinsfile est complet et syntaxiquement correct. Le pipeline s'exécute avec succès du premier coup (les captures d'écran de la console le prouvent).	/ 6
Audit SonarQube	Le Quality Gate est fonctionnel. L'état initial désastreux est bien documenté. L'étudiant a su repérer la faille de sécurité et les Code Smells dans l'interface.	/ 4
Correction du Code	Le code Java a été proprement refactorisé. La faille (mot de passe en dur) est corrigée, le Logger remplace le System.out, et le code mort a disparu.	/ 6
Qualité du livrable	Le rapport est synthétique (pas de remplissage inutile), professionnel et chronologique. Les captures d'écran ciblent exactement ce qui est demandé.	/ 4
Bonus : Conteneurisation	<i>Le fichier Dockerfile est valide et une preuve de l'exécution locale du conteneur est fournie.</i>	+ 2 pts

Grille d'évaluation : Restitution Orale et Démonstration (sur 20 points)

Critère d'évaluation	Description des attentes	Points
Démonstration technique (Live)	L'équipe lance le pipeline en direct avec succès. La navigation dans Jenkins et SonarQube est fluide et maîtrisée. L'effet "démon" est réussi sans accrocs majeurs.	/ 6
Maîtrise et Justifications	Explications claires des choix techniques. L'équipe sait justifier l'impact métier de ses corrections.	/ 6
Dynamique et Temps de parole	La répartition du temps de parole est parfaitement équilibrée. Chaque étudiant prend la main sur une partie technique pertinente. Aucun membre n'est mis à l'écart.	/ 4
Questions / Réponses	L'équipe fait face aux imprévus et répond de manière pertinente aux questions. L'implication est collective (le groupe ne se repose pas sur un seul "leader technique").	/ 4